UNIVERSITÀ DEGLI STUDI DI CATANIA

Dipartimento di Matematica e Informatica
Corso di Laurea Magistrale In Informatica(LM-18)

Francesco Portoghese

Sistemi Cloud

Anno Accademico 2025 - 2026

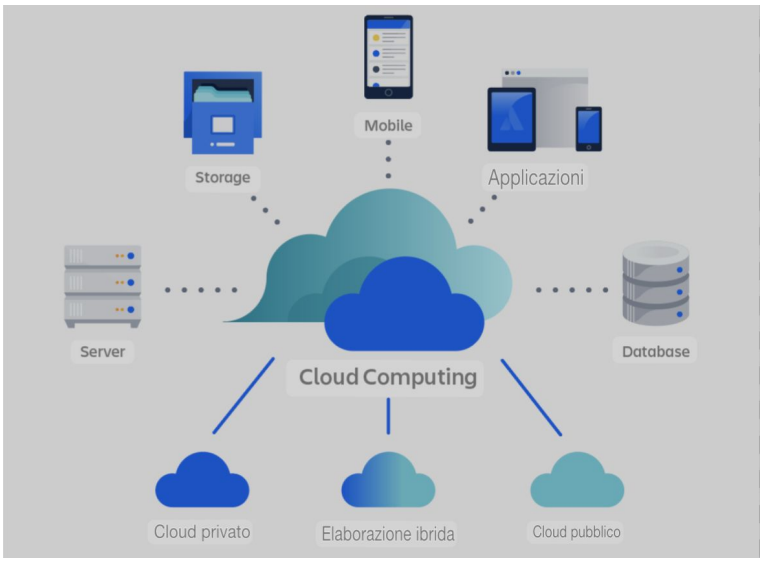
Capitolo 1

Introduzione ai Sistemi Cloud

1.1 Cloud Computing

L'impiego tipico del cloud è quello di gestire un applicazione multi-tier (quotidianamente 3-tier). Le componenti infrastrutturali sono collegate tramite una rete virtualizzata sfruttando le risorse fisiche sicchè:

- l'applicazione gira su host virtuali da una rete virtuale.
- le risorse fisiche si trovano in un pooling per essere condivise.



1.2 Cloud pubblico

Dato che gli host virtuali su cui girano l'app e il DB sono collegati dalla rete virtuale e risiedono nel cloud si dice che l'app è stata **claudificata**. Le risorse fisiche che vengono scambiate tra le parti vengono messe a disposizione da un cloud provider, la quale vende i servizi cloud: ecco che parliamo di **Cloud Pubblico**.

1. **Vantaggi del cloud Pubblico: semplicità d'uso, economicità, sicurezza e innovazione**, tutte caratteristiche che permettono di avere un cost efficency basso, una sicurezza nota e un'innovazione che garantisce soluzioni tecnologiche sempre sul fronte avanzato .

2. **Alcuni Cloud Provider:** AWS(Amazon Web Service), Microsoft Azure,IBM, GoogleCloud

1.3 Cloud Privato

L'infrastruttura è dedicata esclusivamente a una singola organizzazione, può essere gestita internamente o da un provider esterno.

Vantaggi Cloud Privato: **Flessibilità:** in grado di adattarsi a diverse casistiche, **prestazioni:** certe (non esposta a flutttuazioni), **security e compilance:** non in senso assoluto, ma da la possibilità di aderire a standard prescelti dall'organizzazione.

1.4 Cloud Ibrido e Multi-Cloud

Cloud Ibrido rappresenta un modello architetturale in cui vengono combinati i vantaggi del cloud pubblico e del cloud privato. In questo scenario, un'organizzazione può mantenere in un'infrastruttura privata i dati più sensibili o i servizi critici, sfruttando al contempo la scalabilità e la flessibilità del cloud pubblico per gestire carichi di lavoro variabili o progetti temporanei. Ad esempio, un'azienda può conservare le informazioni dei propri clienti in un data center privato per ragioni di compliance, ma utilizzare i servizi di AWS o Azure per eseguire analisi su larga scala quando necessario. I principali **vantaggi del Cloud Ibrido** includono:

- **Flessibilità**: possibilità di distribuire i carichi di lavoro tra pubblico e privato in base alle esigenze.

- **Sicurezza**: dati sensibili rimangono in ambienti controllati, riducendo i rischi di esposizione.
- **Scalabilità**: sfruttamento delle risorse del cloud pubblico per gestire picchi di domanda.
- **Ottimizzazione dei costi**: utilizzo del cloud pubblico solo quando realmente necessario.

Il modello **Multi-Cloud**, invece, si riferisce all'uso simultaneo di più cloud pubblici appartenenti a provider diversi (es. AWS, Microsoft Azure, Google Cloud Platform). Questo approccio è sempre più diffuso perché consente di evitare il fenomeno del *vendor lock-in*, cioè la dipendenza esclusiva da un unico provider.

I principali **vantaggi del Multi-Cloud** sono:

- **Resilienza e continuità operativa**: se un provider ha un disservizio, le applicazioni possono continuare a funzionare sull'altro.
- **Ottimizzazione delle performance**: ogni cloud può essere scelto in base ai servizi in cui eccelle (ad esempio, AWS per l'ampia offerta, GCP per il machine learning, Azure per l'integrazione con sistemi Microsoft).
- **Maggiore potere contrattuale**: riduzione della dipendenza da un singolo fornitore.

In sintesi, il **Cloud Ibrido** punta a unire privato e pubblico per equilibrare sicurezza e flessibilità, mentre il **Multi-Cloud** mira a sfruttare più provider contemporaneamente per aumentare resilienza e ottimizzazione. "‘

1.5 Cloud Computing: Modelli di Servizio

1. **IaaS(Infrastucture as a Service**: Fornisce l'infrastruttura di base - server virtuali, storage, networking. Esempi: Amazon EC2, Google Compute Engine.

2. **PaaS(Platform as a Service)**:Offre una piattaforma di sviluppo completa con runtime, database, strumenti di sviluppo. Esempi: Google App Engine, Microsoft Azure App Service.

3. **SaaS(Software as a Service):**Applicazioni complete accessibili via web. Esempi: Gmail, Salesforce, Microsoft 365.

Capitolo 2

Principi del Cloud-Native

Il paradigma **cloud-native** non si limita a "spostare" un'applicazione nel cloud, ma prevede un insieme di principi architetturali e metodologici che consentono di progettare, sviluppare e gestire applicazioni pensate sin dall'inizio per sfruttare al meglio l'infrastruttura cloud. Tra i concetti chiave rientrano i microservizi, la containerizzazione, le pratiche DevOps, la scalabilità dinamica e il monitoraggio continuo.

2.1 Microservizi e Containerizzazione

I **microservizi** rappresentano un approccio architetturale in cui un'applicazione viene suddivisa in più componenti indipendenti, ognuno responsabile di una funzionalità specifica (ad esempio: autenticazione utenti, gestione pagamenti, registrazione membri). Questa modularità semplifica la manutenzione, permette di sviluppare e distribuire i servizi in maniera indipendente e consente una maggiore resilienza: se un microservizio fallisce, l'intera applicazione non si interrompe. La **containerizzazione** è la tecnologia che consente di impacchettare un microservizio insieme alle sue dipendenze in un ambiente isolato chiamato *container*. Strumenti come **Docker** hanno reso la containerizzazione uno standard de facto, mentre piattaforme come **Kubernetes** gestiscono il deployment e l'orchestrazione dei container, automatizzando scalabilità, bilanciamento del carico e tolleranza ai guasti.

2.2 DevOps e CI/CD

Il movimento **DevOps** unisce sviluppo (*Dev*) e operazioni (*Ops*) con l'obiettivo di ridurre il tempo che intercorre tra lo sviluppo di una nuova funzionalità e la sua disponibilità in produzione. Le pratiche DevOps promuovono la collaborazione tra team, l'automazione dei processi e la cultura del miglioramento continuo. Una delle implementazioni concrete di DevOps è la **CI/CD (Continuous Integration / Continuous Delivery)**. La **CI** prevede che ogni modifica al codice venga integrata frequentemente e testata automaticamente, mentre la **CD** assicura che tali modifiche possano essere rilasciate in maniera automatizzata e affidabile in produzione. In questo modo si ottiene un ciclo di sviluppo più rapido, con minor rischio di errori umani e maggiore qualità del software.

2.3 Scalabilità: Orizzontale e Verticale

Uno dei vantaggi principali delle architetture cloud-native è la capacità di adattarsi dinamicamente alla domanda. Esistono due tipi di **scalabilità**:

- **Scalabilità verticale**: consiste nell'aumentare le risorse (CPU, RAM, storage) di una singola macchina. È un approccio semplice da implementare ma presenta limiti fisici e può risultare costoso.
- **Scalabilità orizzontale**: consiste nell'aggiungere più istanze di un servizio, distribuendo il carico tra più nodi. Questo approccio è altamente efficace in ambienti cloud, poiché consente di adattarsi ai picchi di traffico aggiungendo o rimuovendo istanze in maniera automatica tramite sistemi di auto-scaling.

In contesti moderni, la scalabilità orizzontale è preferita perché garantisce resilienza, bilanciamento del carico e riduzione del rischio di singoli punti di fallimento.

2.4 Osservabilità e Monitoraggio nel Cloud

La **osservabilità** è la capacità di comprendere lo stato interno di un sistema complesso a partire dalle informazioni generate dalle sue operazioni. Essa va oltre il semplice **monitoraggio**, poiché non si limita a

rilevare se un servizio è attivo o meno, ma consente di analizzare in profondità performance, anomalie e cause di eventuali malfunzionamenti. Gli strumenti di osservabilità tipici includono:

- **Metriche**: valori numerici che descrivono lo stato del sistema (CPU usage, latenza, throughput).
- **Log**: registri testuali che tracciano eventi e operazioni interne.
- **Traces**: tracciamenti che seguono il percorso di una richiesta tra diversi microservizi, utili per individuare colli di bottiglia.

Piattaforme come **Prometheus**, **Grafana**, **ELK Stack** (Elasticsearch, Logstash, Kibana) e i servizi nativi del cloud (es. **AWS CloudWatch**, **Azure Monitor**) consentono di implementare monitoraggio e osservabilità efficaci, migliorando l'affidabilità e l'esperienza utente finale.

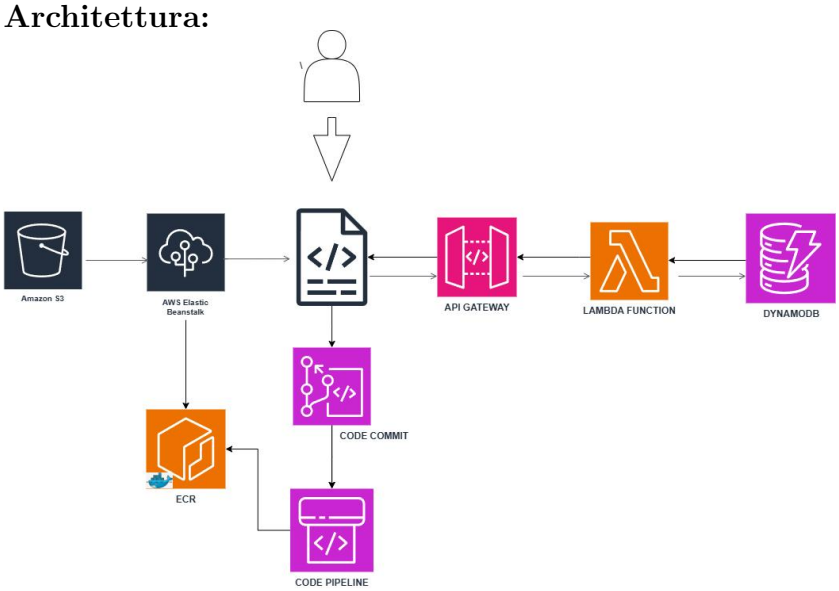
Capitolo 3

GymCloud

3.1 BODY POWER GYM

Body Power Gym è il nome del progetto Cloud realizzato usando i servizi del Cloud Provider AWS. Il **GOAL** dell'applicazione consiste nel poter registrare tutti i nuovi atleti(facendo delle chiamati API Gatway) inscritti in palestra, inserendo nome, cognome,e-mail,numero di telefono(necessarimente di 10 cifre) ,l'opportunità di selezione uno o piu mesi per l'inscrizione e l'obbiettivo che si vuole raggiungere, ovviamente usando i servizi che AWS ci mette a disposizione.

– **Architettura:**



- **Servizi AWS usati:**
  * **S3(Simple Storage Service)**: Un "magazzino digitale" dove puoi conservare qualsiasi tipo di file (foto, video, documenti, backup). È come Google Drive ma per sviluppatori e aziende.
  * **AWS Elastick Benkstal**:Ti permette di pubblicare la tua applicazione web senza preoccuparti dei server. Carichi il codice e AWS si occupa di tutto il resto (server, bilanciamento del carico, scalabilità).
  * **ECR,Docker**:Un "deposito" per le immagini Docker. Come Docker Hub ma privato e integrato con gli altri servizi AWS.
  * **Code Commit**: Come GitHub ma di Amazon. Un posto dove conservare e versionate il codice sorgente dei tuoi progetti.
  * **Code Pipeline**:Automatizza il processo di deployment. Quando modifichi il codice, questo servizio può automaticamente testarlo e pubblicarlo in produzione.
  * **API Gatway**: Un "centralino" per le tue API. Gestisce le chiamate alle tue funzioni, controlla chi può accedervi e monitora il traffico.
  * **Lambda Function**: Esegue il tuo codice senza dover gestire server. Paghi solo quando il codice viene effettivamente eseguito - perfetto per operazioni sporadiche.
  * **Dynamo DB**:Un database NoSQL velocissimo, completamente gestito da AWS. Ideale per applicazioni che hanno bisogno di accesso rapido ai dati su larga scala.

Tutti questi servizi lavorano insieme per creare applicazioni moderne e scalabili nel cloud.

3.2 Sicurezza e IAM in BODY POWER GYM

La sicurezza è un aspetto fondamentale in qualsiasi applicazione cloud, soprattutto quando vengono trattati dati personali degli utenti, come nel caso di BODY POWER GYM. Un'architettura sicura non si limita alla protezione dei dati, ma comprende anche

la gestione delle identità, dei permessi e delle pratiche di conformità. In AWS, la sicurezza viene implementata attraverso un modello di **responsabilità condivisa**: AWS garantisce la protezione dell'infrastruttura fisica e dei servizi di base, mentre l'utente (o l'organizzazione) è responsabile della configurazione sicura delle proprie risorse.

Identity and Access Management (IAM)

IAM (Identity and Access Management) è il servizio di AWS che consente di gestire utenti, gruppi e ruoli, definendo in maniera granulare chi può accedere a quali risorse e con quali privilegi. In BODY POWER GYM, IAM viene utilizzato per garantire che ogni componente dell'applicazione disponga solo dei permessi strettamente necessari al proprio funzionamento (*principio del privilegio minimo*). Esempi di configurazioni IAM in BODY POWER GYM:

- Le **Lambda Function** hanno permessi limitati: ad esempio, possono scrivere e leggere su DynamoDB, ma non hanno accesso diretto ad altri servizi non necessari.
- Il **bucket S3** che ospita il frontend è configurato con policy di accesso pubblico in sola lettura, così che i file statici possano essere distribuiti via web, mantenendo al contempo bloccate le scritture non autorizzate.
- Gli sviluppatori accedono al codice su **CodeCommit** tramite credenziali IAM sicure e, se possibile, con autenticazione a più fattori (MFA).

Best Practice di Sicurezza in BODY POWER GYM

Oltre a IAM, BODY POWER GYM adotta altre misure di sicurezza tipiche del cloud:

- **Crittografia dei dati**: i dati in DynamoDB sono cifrati sia a riposo che in transito.
- **Gestione delle chiavi (KMS)**: le chiavi crittografiche vengono gestite tramite AWS KMS, garantendo rotazione e controllo centralizzato.

- **Logging e Audit**: ogni operazione sugli utenti e sulle risorse è tracciata tramite AWS CloudTrail, permettendo di individuare eventuali accessi sospetti.
- **Sicurezza di rete**: le comunicazioni avvengono su HTTPS, con certificati SSL gestiti da AWS Certificate Manager.

Grazie a queste misure, BODY POWER GYM riesce a garantire la protezione dei dati degli atleti, la conformità alle buone pratiche di sicurezza e un accesso controllato ai servizi cloud.

3.3 Goal Del Progetto

L'idea è quella di fare una **"GYMCloud"** (un apllicazione sulla palestra). La parte frontend viene gestita da **S3 e Elastick Benkstal** , l'utente fa delle richieste e il browser chiama **API Gatway, Lambda Function** esegue un compito specifico in risposta alla richiesta api e quando la Lambda Function ha bisogno di leggere o scrivere dati interagisce con **DynamoDB.** Ogni modifica fatta viene salvato in **code commit** e ogni volta che viene effettuato un "commit", o quando una nuova immagine viene spinta su **ECR**, **Code Pipeline** viene attivato automaticamente. ECR contiene le immagini **Docker** e una volta che la nuova immagine Docker è su ECR, Code Pipeline invierà un comando a elastick benkstal per aggiornare l'applicazione. Elastic Beanstalk, a sua volta, preleverà la nuova immagine Docker da ECR e la distribuirà sui suoi ambienti.

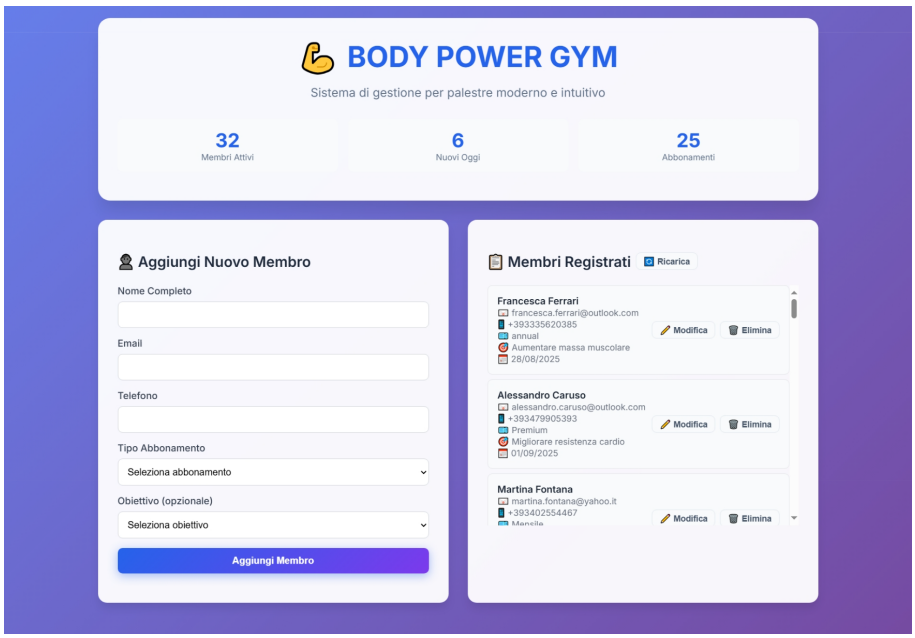
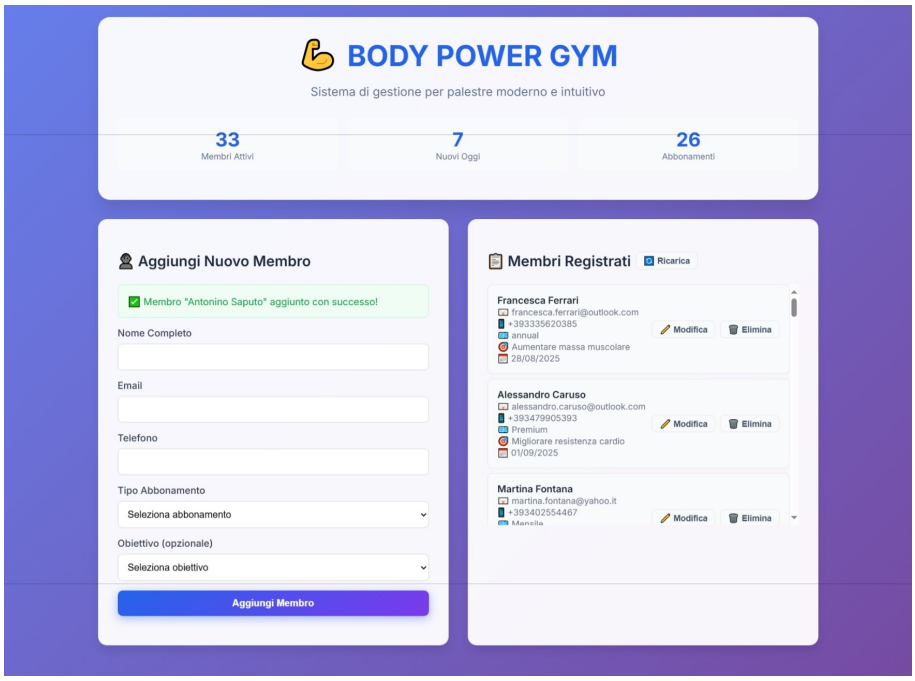


Figura 3.1: Enter Caption

3.4 Test Applicazione



Dalle due foto si evince come funziona l'applicazione, ogni persona può inscriversi in palestra basta inserire come detto pocansi nome,cognome,e-mail,numero di telefono ,tipologia di abbonamento e sopratutto per ultimo ma non per meno importanza, l'obbiettivo che si vuole ottenere. Alla fine dell'inscrizione l'utente verrà **"AGGIUNTO CON SUCCESSO!** La pagina web è sempre aggiornata, infatti tutte le persone inscritte in palestra sono salvate nel Data Base in ordine di inscrizione. Oltre a salvare tutti i membri della palestra nel Data Base, la pagina web ci dice pure le persone inscritte in data odierna, tenendoci sempre aggiornati sui membri, quindi se il giorno dopo riavviamo l'app e non abbiamo nessun inscritto i **"Nuovi Membri"** saranno 0, come giusto che sia. La pagina oltre a dirci i **MEMBRI ATTIVI** e i **NUOVI OGGI** ci da pure il numero di **ABBONAMENTI**. "'latex

Capitolo 4

Estensioni Future del Progetto

BODY POWER GYM, pur essendo già funzionale e scalabile, offre ampie possibilità di evoluzione. Le seguenti estensioni delineano come l'applicazione potrebbe crescere in futuro, integrando nuove tecnologie e migliorando l'esperienza degli utenti.

4.1 Integrazione con Wearable Devices

L'integrazione con dispositivi indossabili (*wearable devices*) come smartwatch e fitness tracker permetterebbe di raccogliere dati biometrici in tempo reale, quali frequenza cardiaca, calorie bruciate e qualità del sonno. Queste informazioni potrebbero essere sincronizzate con BODY POWER GYM per offrire un'esperienza più personalizzata, monitorando i progressi degli atleti e suggerendo piani di allenamento adattivi. Inoltre, i trainer avrebbero accesso a dati più accurati per migliorare il supporto agli iscritti.

4.2 Analisi Dati e Intelligenza Artificiale

Un'estensione naturale di BODY POWER GYM riguarda l'utilizzo di **Big Data** e **Intelligenza Artificiale (AI)**. Grazie

ai dati raccolti dalle sessioni di allenamento, dagli accessi in palestra e dai dispositivi indossabili, sarebbe possibile implementare algoritmi di machine learning per:

* generare raccomandazioni personalizzate per gli allenamenti;
* prevedere il rischio di infortuni analizzando i pattern di esercizio;
* ottimizzare l'utilizzo delle attrezzature e degli spazi in palestra;
* segmentare gli utenti per campagne di marketing mirate.

L'AI trasformerebbe BODY POWER GYM in una piattaforma intelligente, capace non solo di gestire, ma anche di supportare attivamente l'evoluzione del percorso fitness degli utenti.

4.3 Gamification e Fidelizzazione degli Utenti

Per aumentare il coinvolgimento degli iscritti, BODY POWER GYM potrebbe adottare meccaniche di **gamification**. Esempi includono:

* badge e riconoscimenti per il raggiungimento di obiettivi;
* classifiche e sfide tra utenti per stimolare la motivazione;
* premi digitali o sconti sugli abbonamenti per chi mantiene costanza negli allenamenti.

Queste strategie non solo migliorano l'esperienza dell'utente, ma contribuiscono anche a rafforzare la fidelizzazione e a ridurre il tasso di abbandono.

4.4 Possibili Migliorie Architetturali

Dal punto di vista tecnico, BODY POWER GYM potrebbe evolvere ulteriormente adottando:

* **Architettura serverless avanzata**: riducendo ulteriormente la gestione dei server e sfruttando al massimo i servizi on-demand.

* **Approccio multi-cloud**: distribuendo i carichi di lavoro su diversi provider per aumentare resilienza e ridurre i rischi di lock-in.
* **Edge computing**: elaborazione dei dati più vicino agli utenti e ai dispositivi indossabili, per ridurre latenza e migliorare le performance.
* **Maggiore automazione nella sicurezza**: con strumenti di *Security as Code* per garantire la compliance continua.

Queste migliorie consentirebbero a BODY POWER GYM di consolidarsi come piattaforma moderna, resiliente e innovativa, capace di adattarsi alle esigenze future del mercato del fitness digitale. “‘

Capitolo 5

Conclusioni Personali sul Progetto Sviluppato

Questo progetto ha una funzionalità importante, ti permette di avere delle conoscenze sui vari servizi di AWS, che non sono scontate. Il progetto utilizza un approccio cloud-native molto ben strutturato, combinando servizi AWS managed (S3, Beanstalk, API Gateway, Lambda, DynamoDB) con pratiche DevOps moderne (ECR, pipeline CI/CD). Questa scelta indica una mentalità orientata alla scalabilità e alla manutenibilità. La presenza sia di AWS Beanstalk (per applicazioni containerizzate) che di Lambda Functions suggerisce un approccio ibrido intelligente: Lambda gestisce operazioni specifiche e legge, mentre Beanstalk ospita la logica applicativa principale. Questo equilibrio è spesso ottimale per performance e costi. L'uso di DynamoDB per il database NoSQL combinato con S3 per lo storage indica una strategia di gestione dati moderna, orientata a gestire sia dati strutturati che non strutturati (file, media, documenti). Il flusso che va da Code Commit → Code Pipeline → ECR mostra un'implementazione solida di CI/CD, essenziale per deployment frequenti e affidabili. Nel complesso, è un'architettura ben pensata che bilancia modernità, scalabilità e praticità operativa.